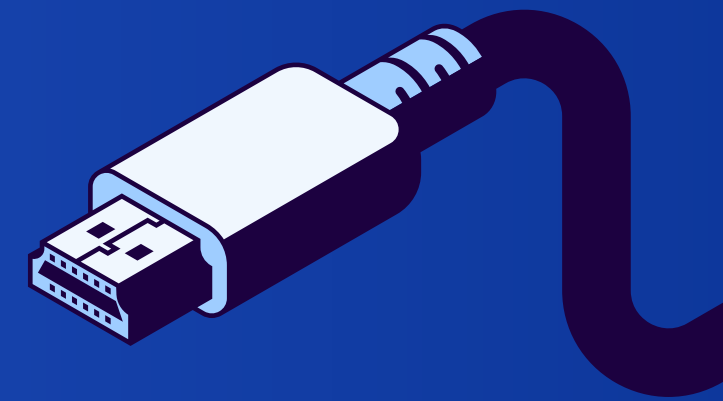
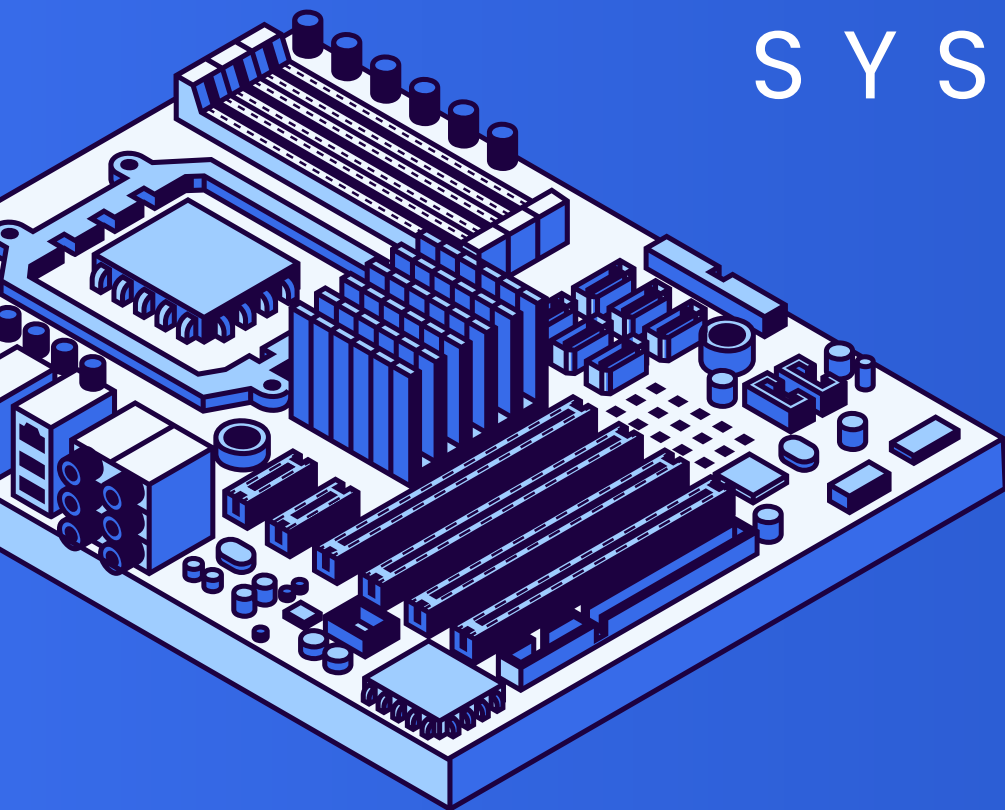


To-Do List App

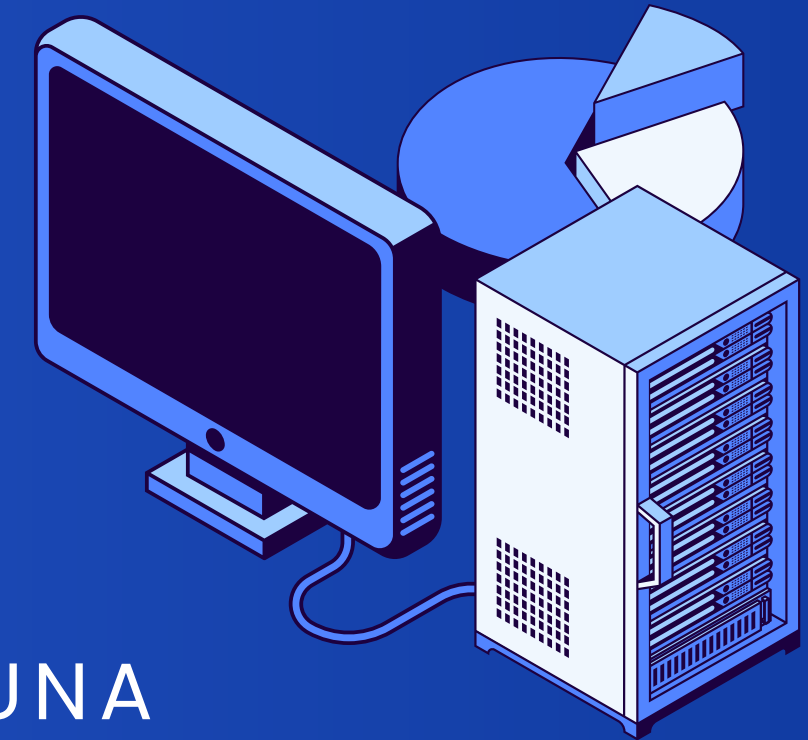


SYSTEM DESIGN CASE STUDY



MASALAH

Banyak orang kesulitan mengatur tanggung jawab sehari-hari karena tugas dari sekolah, pekerjaan, dan kehidupan pribadi terus menumpuk. Catatan kertas dan pengingat manual tidak mampu mengikuti jadwal yang cepat berubah, dan belum ada cara mudah untuk menentukan prioritas berdasarkan deadline atau tingkat urgensi. Akibatnya, kegiatan hobi pun sering terlupakan karena tercampur dengan kewajiban.

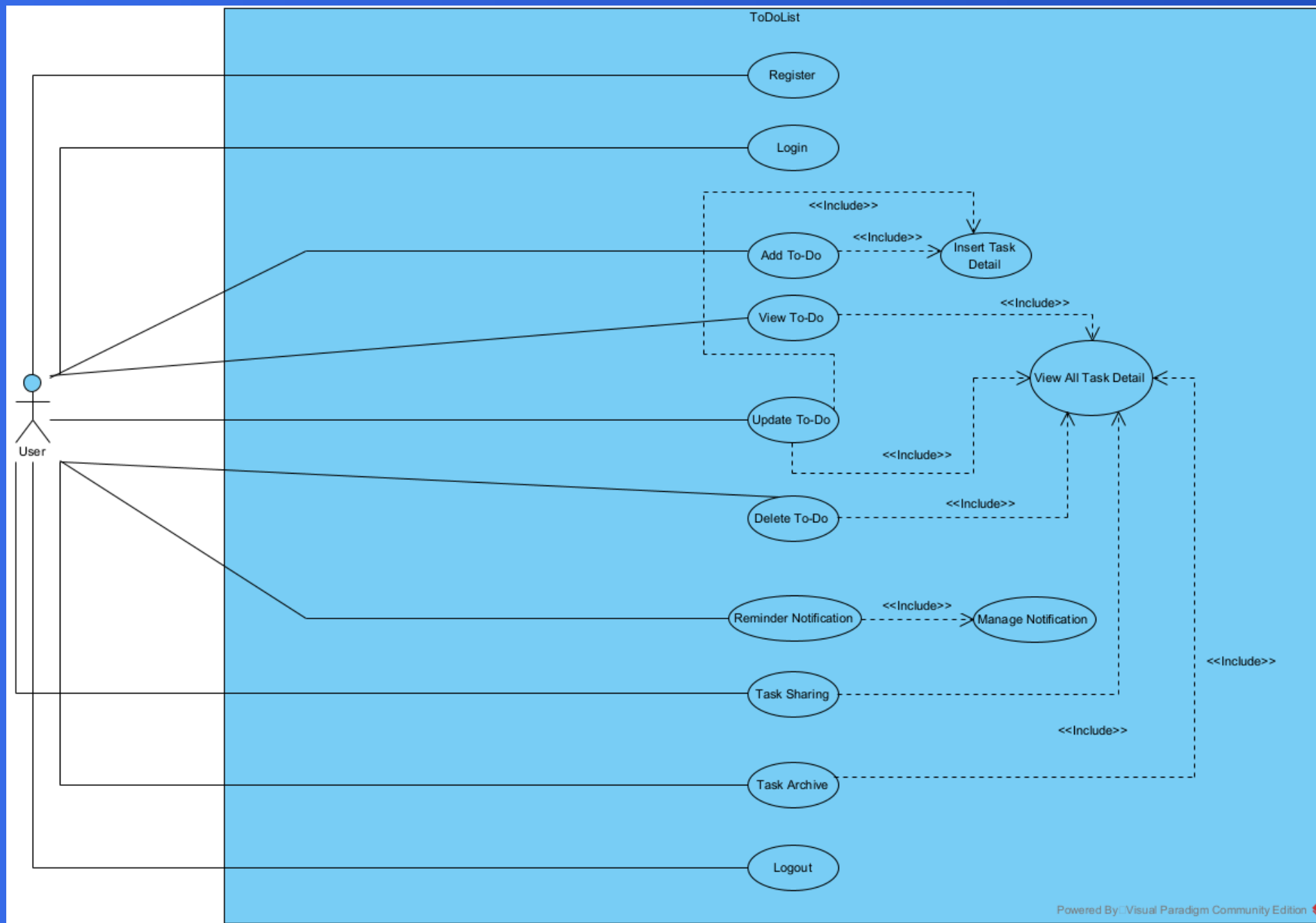


TARGET PENGGUNA

Individu yang ingin mengatur waktu dengan lebih baik: pelajar yang punya banyak tugas, profesional muda yang mengelola pekerjaan, dan siapa saja yang ingin menjadwalkan hobi di samping kewajibannya. Aplikasi ini ditujukan untuk penggunaan pribadi, bukan untuk tim atau perusahaan.

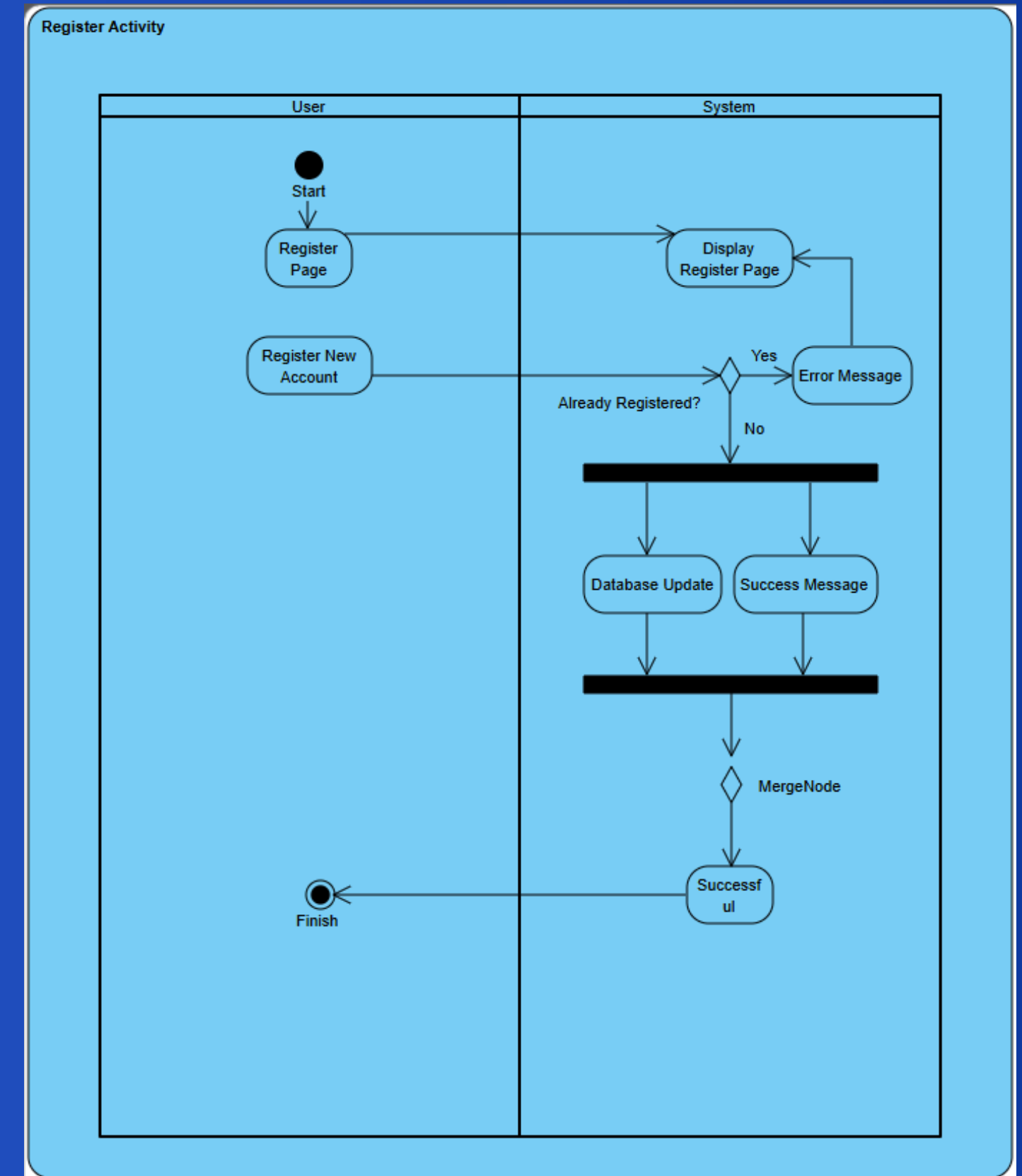
Use case diagram: satu aktor, 10 fungsi utama, dan 3 sub- fungsi bersama

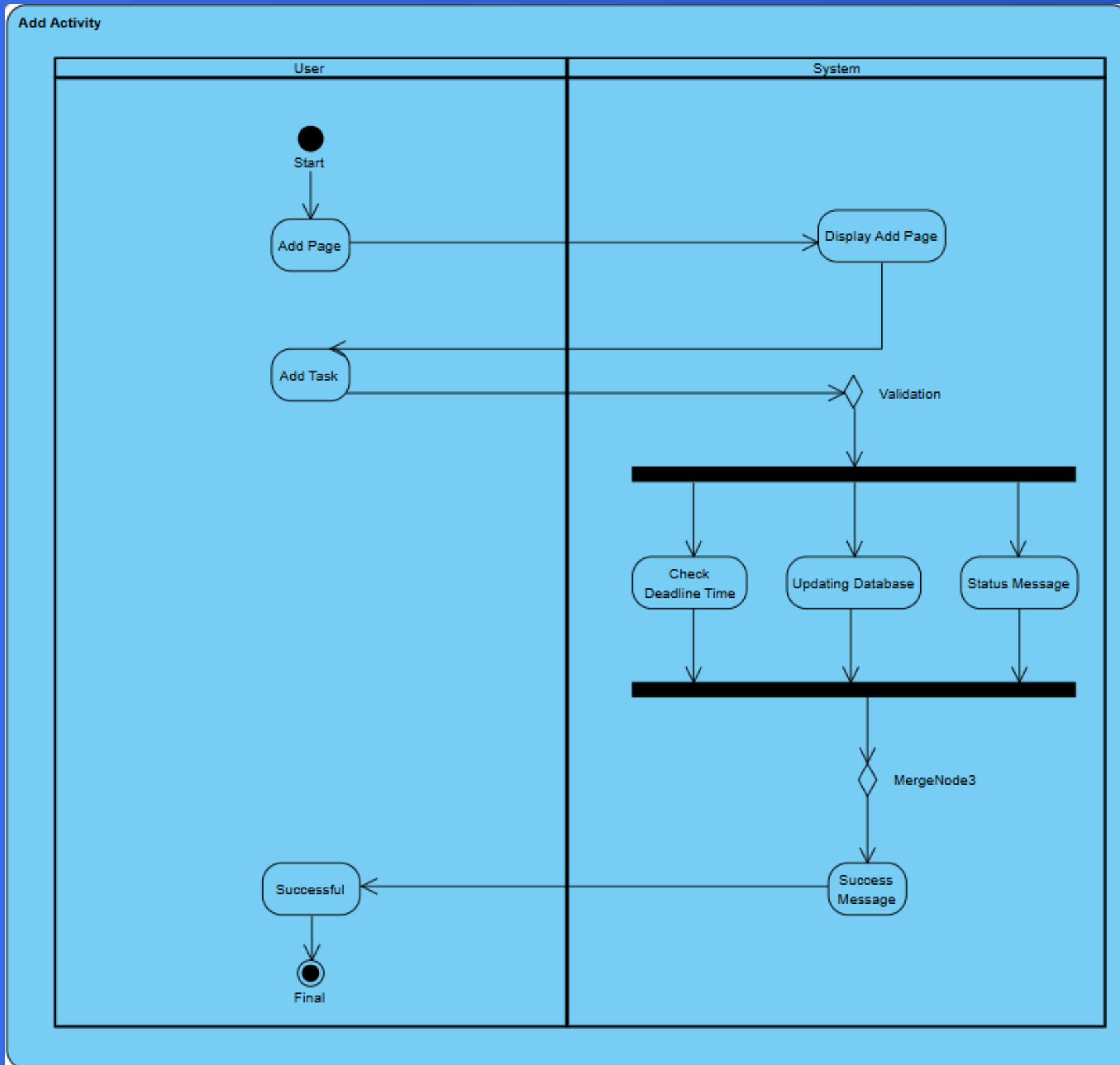
Sistem memiliki satu aktor, User, yang dapat mengakses sepuluh fungsi dalam tiga kelompok: Account (Register, Login, Logout), Task Management (Add, View, Update, Delete), dan Productivity (Reminder, Sharing, Archive).



Alur Register dengan swimlane User dan System

Swimlane memisahkan tindakan pengguna dan tindakan sistem. Decision node memeriksa apakah username sudah terdaftar. Jika sudah, pengguna mendapat pesan error dan kembali ke form. Jika belum, fork menjalankan update database dan pesan sukses secara paralel, lalu alur selesai.



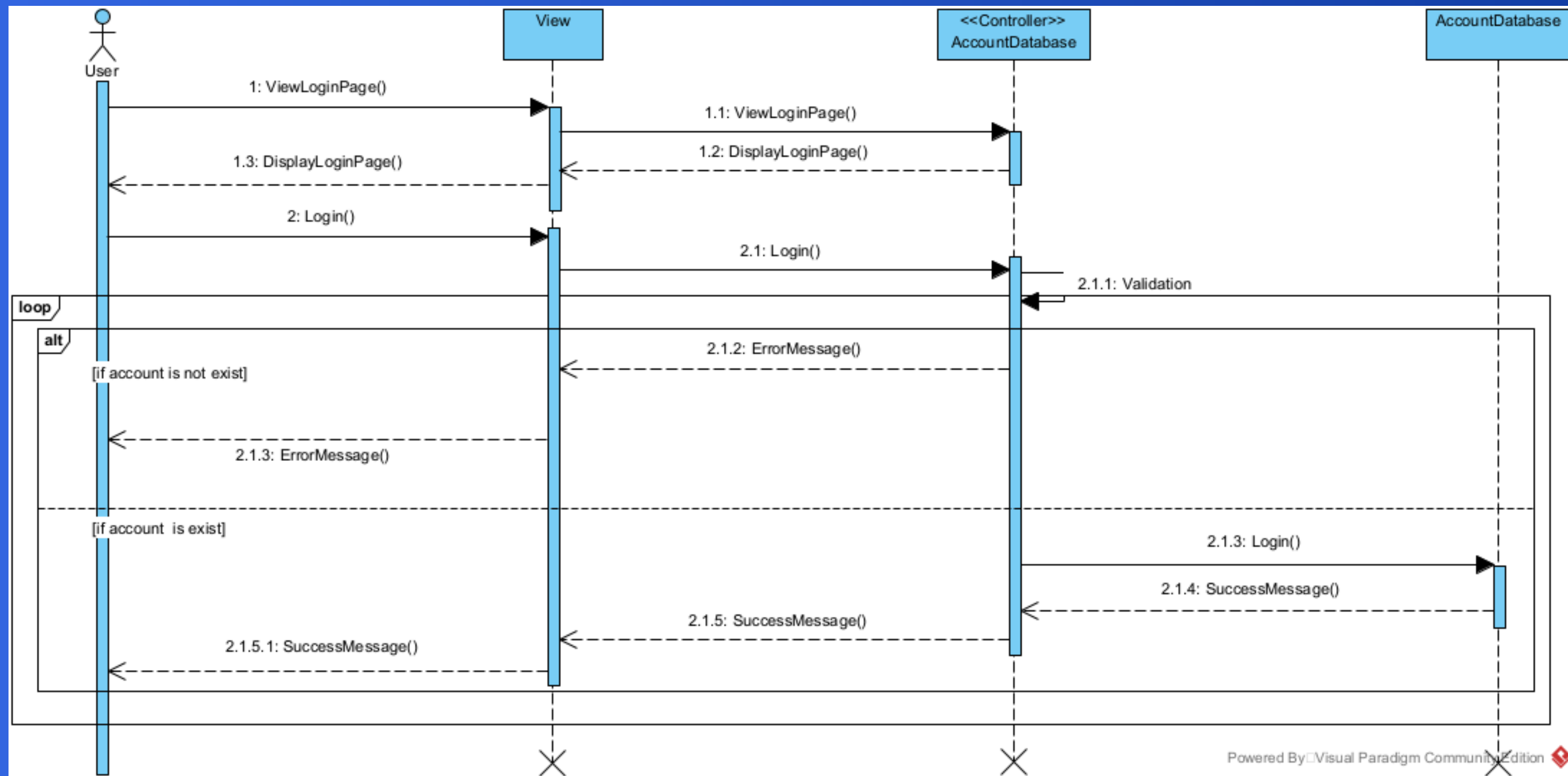


Alur Add Task dengan tiga proses paralel setelah validasi

Setelah pengguna mengirim tugas, sistem memvalidasi data lalu menjalankan tiga proses independen secara bersamaan: memeriksa deadline, menyimpan ke database, dan menyiapkan pesan status. Join menunggu ketiganya selesai sebelum menampilkan pesan sukses.



Sequence Login: View → Controller → Database, dengan validasi dan penanganan error



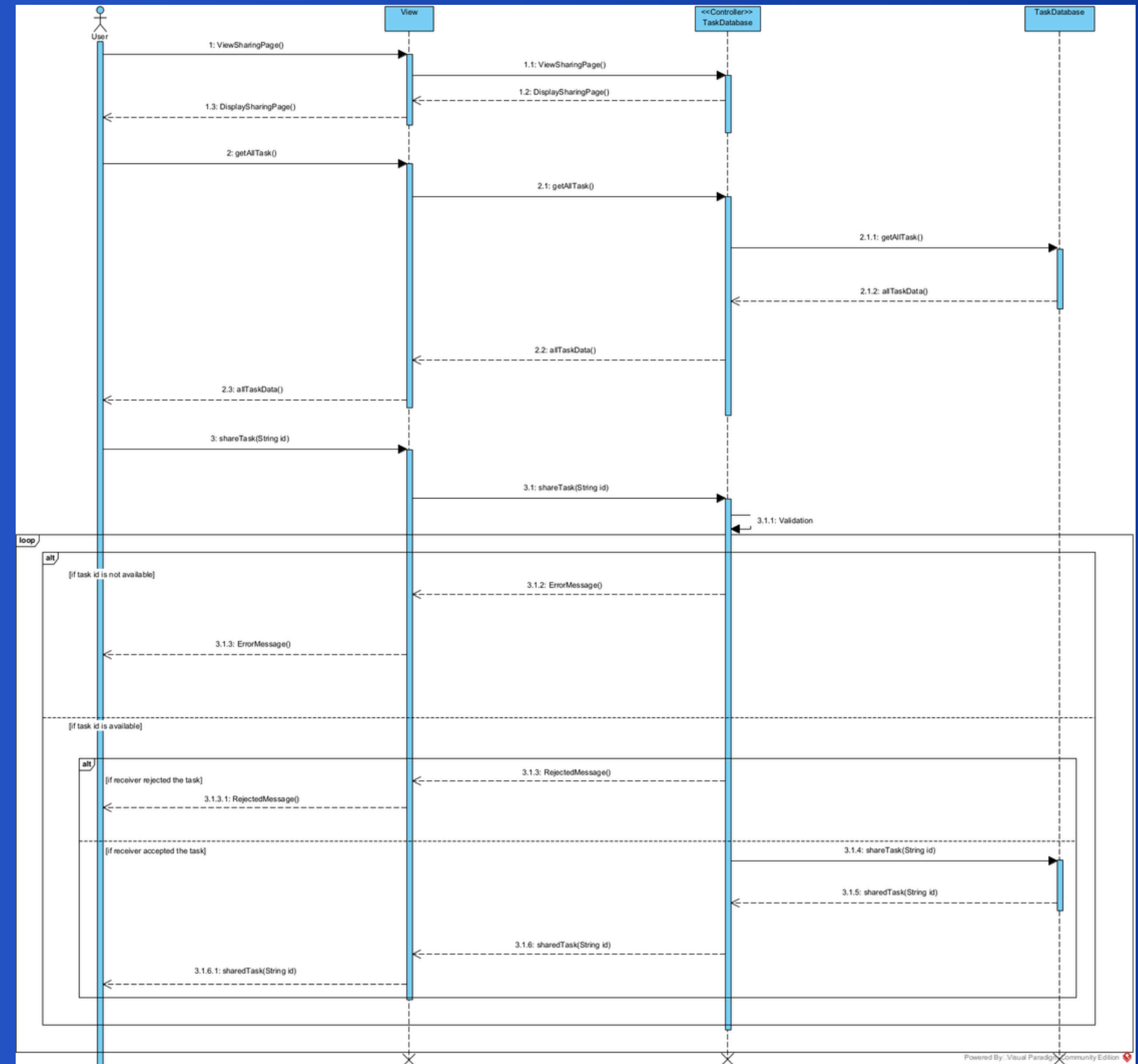
Sequence Login: View → Controller → Database, dengan validasi dan penanganan error

Kesepuluh sequence diagram mengikuti pola yang sama. View menerima input dan meneruskannya ke Controller untuk divalidasi. Fragment alt membagi hasilnya: pesan error jika akun tidak ada, atau pengecekan ke database dan pesan sukses jika ada. Seluruh proses berada di dalam loop, sehingga pengguna bisa mencoba lagi.



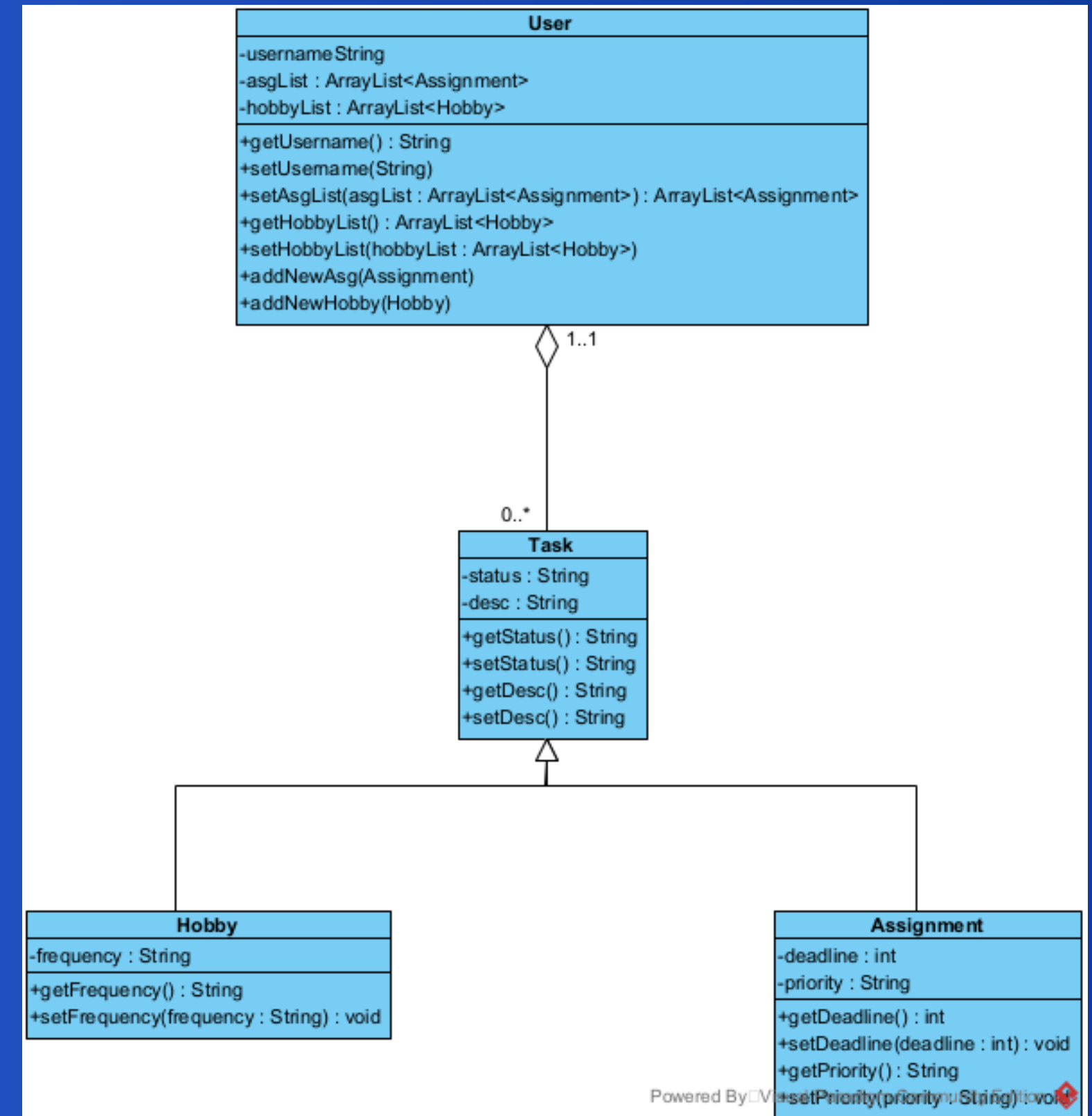
Task Sharing, alur paling kompleks dengan keputusan bertingkat

Proses sharing terdiri dari tiga tahap: membuka halaman sharing, memuat semua tugas, lalu membagikan satu tugas berdasarkan ID. **alt** pertama memeriksa apakah tugas tersedia. Di dalamnya, **alt** kedua menangani respons penerima: menolak, atau menerima sehingga data sharing disimpan. Ini satu-satunya alur yang melibatkan pihak kedua.



Class diagram

Task adalah class abstrak yang berisi atribut bersama semua tugas: status dan desc. Assignment menambahkan deadline dan priority, sedangkan Hobby menambahkan frequency. Satu User dapat memiliki nol atau lebih tugas. Semua atribut bersifat private dan diakses melalui getter dan setter. Dengan struktur ini, tipe tugas baru bisa ditambahkan tanpa mengubah class yang sudah ada.



Factory Pattern: membuat tugas tanpa bergantung pada tipenya

Pembuatan objek dipisahkan dari logika aplikasi. App cukup memilih factory yang sesuai, AssignmentFactory atau HobbyFactory, lalu memanggil createTask(). Jika nanti ada tipe tugas baru, cukup tambahkan satu factory baru tanpa mengubah kode yang sudah ada.

```
Factory Pattern

factories/TaskFactory.java
public interface TaskFactory {
    Task createTask(User user, String status, String desc, String additionalInfo);
}

factories/HobbyFactory.java
public class HobbyFactory implements TaskFactory {
    @Override
    public Task createTask(User user, String status, String desc, String additionalInfo) {
        return new Hobby(user, status, desc, additionalInfo);
    }
}

app/App.java · addTask()
TaskFactory factory;
if (type.equalsIgnoreCase("Assignment")) {
    // ...read deadline and priority
    factory = new AssignmentFactory();
} else if (type.equalsIgnoreCase("Hobby")) {
    // ...read frequency
    factory = new HobbyFactory();
}
Task newTask = factory.createTask(currentUser, status, desc, additionalInfo);
currentUser.addTask(newTask);
```

Strategy Pattern: aksi yang dapat dipertukarkan

Update status dan hapus tugas diimplementasikan sebagai strategi terpisah dari satu interface, TaskModificationStrategy. App hanya memanggil modify() tanpa perlu tahu detail aksinya, sehingga aksi baru seperti archive bisa ditambahkan dengan membuat strategi baru.

```
strategies/TaskModificationStrategy.java
public interface TaskModificationStrategy {
    void modify(Task task, String[] params);
}

strategies/UpdateStatusStrategy.java
public class UpdateStatusStrategy implements TaskModificationStrategy {
    @Override
    public void modify(Task task, String[] params) {
        task.setStatus(params[0]);
    }
}

strategies/DeleteTaskStrategy.java
public class DeleteTaskStrategy implements TaskModificationStrategy {
    @Override
    public void modify(Task task, String[] params) {
        task.getUser().getTasks().remove(task);
    }
}
```

Implementasi: aplikasi console Java 11

Rancangan UML diimplementasikan menjadi aplikasi console dengan Java 11 di Eclipse. Kode dibagi ke dalam package models, factories, strategies, dan app. Contoh di samping: membuat akun, menambah Assignment, melihat daftar tugas, lalu memperbarui status.

```
To-Do List App - Java 11 console

1. Login 2. Create User 3. Exit
Select an option: 2
Enter new username: eugenia
User created and logged in as: eugenia
...
Logged in as: eugenia
1. Add Task 2. Update Task 3. Delete Task 4. View Tasks 5. Log Out
Select an option: 1
Enter task type (Assignment or Hobby):
Assignment
Enter task description:
Finish UML report
Enter initial status:
ongoing
Enter the deadline in number of days (e.g., 3 for 3 days from now):
3
Enter the priority (e.g., high, medium, low):
high
Task added successfully.
...
Select an option: 4
Your Tasks:
1. Finish UML report (Status: ongoing, Info: 3 days | high)
2. Play guitar (Status: ongoing, Info: weekly)
...
Select an option: 2
Select a task:
1: Finish UML report (Status: ongoing)
2: Play guitar (Status: ongoing)
3: Cancel
1
Current status: ongoing
Enter new status (e.g., ongoing, completed):
completed
Task status updated successfully.
```

Hasil & Refleksi

YANG BERHASIL DIBANGUN

- Menu akun: Create User, Login, Log Out, Exit
- Kelola tugas: Add, View, Update Status, Delete (dengan konfirmasi)
- Dua tipe tugas: Assignment dan Hobby
- Validasi input deadline
- Factory & Strategy Pattern

YANG SAYA PELAJARI

- Menyelaraskan use case, activity, dan sequence diagram memudahkan menemukan celah sebelum coding.
- Design pattern membuat kode mudah ditambah tanpa mengubah bagian lain.
- Membatasi scope membantu fitur inti selesai tepat waktu.

Thank You

EUGENIA LAELLISA HIBAU